

# Airy雷达在已知地图中的定位技术方案

## 1. 需求定义

需求1：全局定位（Global Localization）

问题描述：

- 输入：完整地图（PLY格式）+ 单帧Airy扫描
- 约束：完全不知道扫描在地图中的位置
- 目标：找到扫描在地图中的6DOF位姿

挑战：

- 地图范围大（可能数百米×数百米）
- 单帧扫描覆盖范围小（半径~50-80米）
- 需要在整个地图中搜索

需求2：局部优化（Local Refinement）

问题描述：

- 输入：完整地图（PLY格式）+ 单帧Airy扫描 + 粗略位姿
- 约束：已知大概位置，但误差3-10米
- 目标：精确到 $\pm 0.1$ 米级别

挑战：

- 初值误差较大
- 可能存在动态物体（人、车辆）
- Airy固态雷达点云稀疏

## 2. 技术方案

2.1 需求1的解决方案：Scan Context + TEASER/Quatro

方案架构：

阶段1：全局粗定位（Scan Context）

输入：单帧扫描 + SC数据库（离线预建）

输出：Top-K候选位置（K=3-5）

精度： $\pm 5-10$ 米

↓

## 阶段2：多假设验证（TEASER/Quatro）

对每个候选位置：

1. 提取局部地图（半径50米）
2. FPFH特征提取和匹配
3. TEASER/Quatro鲁棒配准
4. 评估配准质量



## 阶段3：结果选择

- 选择配准质量最好的候选
- 可选：多帧验证

输出：精确6DOF位姿

精度：±0.1-0.2米

## 为什么需要Scan Context？

- TEASER/Quatro需要点对应关系
- 对应关系依赖于两个点云有重叠
- 不知道位置 = 不知道提取地图的哪部分
- SC解决"在哪个区域"的问题

## 流程详解：

### 1. 离线准备（一次性）

- 从地图中每隔5-10米采样一个位置
- 提取该位置周围的局部点云
- 计算Scan Context描述子
- 建立SC数据库

### 2. 在线定位

#### 步骤1：SC检索

- 输入单帧扫描
- 计算其SC描述子
- 在数据库中检索最相似的K个位置

#### 步骤2：对每个候选

- 以候选位置为中心，提取局部地图
- 单帧扫描 vs 局部地图 → FPFH特征匹配
- TEASER/Quatro配准
- 记录配准质量（inlier数量、残差）

#### 步骤3：选择最优

- 比较所有候选的配准质量
- 选择质量最好的作为最终结果

关键参数：

- Scan Context:
- SC候选数量：3-5个
  - 距离阈值：0.3
- FPFH:
- normal\_radius: 0.5-1.0米（根据Airy点密度）
  - fpfh\_radius: 1.0-2.0米
- TEASER/Quatro:
- noise\_bound: 0.05米（Airy精度）
  - 使用Quatro（煤矿场景roll/pitch≈0）

2.2 需求2的解决方案：直接TEASER/Quatro

方案架构：



↓

#### 步骤4: Quatro鲁棒配准

- 输入对应关系 (可能50%以上是错的)
- Quatro自动剔除outliers
- 估计4DOF (x, y, z, yaw)
- roll和pitch从IMU获取或设为0

↓

输出: 精确6DOF位姿  
精度:  $\pm 0.05 - 0.1$ 米,  $\pm 0.5$ 度

### 为什么不需要SC?

- 已知粗略位置 → 知道提取地图的哪部分
- 误差3-10米 → 局部地图50米半径足够覆盖
- 直接FPFH匹配 + TEASER即可

### 为什么用Quatro而非TEASER++?

- 煤矿场景: 地面机器人, roll/pitch $\approx 0$
- Quatro: 4DOF优化, 速度更快
- 可以结合IMU提供的roll/pitch

### 关键参数:

#### 局部地图提取:

- 半径: 粗略误差 + 扫描范围 + 安全余量  
例如: 10米误差 + 50米扫描 + 20米余量 = 80米

#### FPFH:

- normal\_radius: 0.5-1.0米
- fpfh\_radius: 1.0-2.0米

#### Quatro:

- noise\_bound: 0.05米
- rotation\_estimation: QUATRO
- inlier\_selection: PMC\_HEU

---

## 3. Airy固态雷达的特殊处理

### 3.1 点云稀疏问题

#### 问题:

- Airy是非重复扫描，单帧点云稀疏
- FPFH需要足够的邻域点才能计算可靠

解决方案：

#### 方案A：累积多帧（推荐）

```
// 如果是连续扫描场景
累积1-2秒的点云（约10-20帧）
↓
使用IMU去运动畸变
↓
形成更密集的点云
↓
再进行FPFH提取
```

#### 方案B：单帧处理

```
// 如果必须单帧
调整FPFH参数：
- 增大normal_radius和fpfh_radius
- 例如：1.0米和2.0米（而非标准的0.02和0.04）
```

### 3.2 特征提取优化

```
FPFH参数调优（针对Airy）：
- normal_radius: 1.0米（Airy点稀疏，需要大邻域）
- fpfh_radius: 2.0米
- 降采样: 0.5米（减少计算量）
- 去除地面点（可选，提高特征区分度）
```

PROF

## 4. 实现路线图

### 4.1 Example 1: 需求2（局部优化）

先实现这个，因为：

- 更简单（不需要SC数据库）
- 可以直接验证TEASER/Quatro效果
- 为需求1打基础

输入准备：

1. map.ply - 全局地图
2. scan.pcd - 单帧Airy扫描（或累积点云）
3. rough\_pose.txt - 粗略位姿  
格式: x y z roll pitch yaw

输出：

1. refined\_pose.txt - 精确位姿
2. alignment\_quality - 配准质量指标
3. 可视化：配准前后对比

## 4.2 Example 2: 需求1（全局定位）

基于Example 1扩展：

- 增加SC数据库构建
- 增加SC检索步骤
- 增加多候选验证逻辑

输入准备：

1. map.ply - 全局地图
2. scan.pcd - 单帧Airy扫描
3. (可选) sc\_database.bin - 预建SC数据库

输出：

1. estimated\_pose.txt - 估计位姿
2. candidates.txt - 所有候选及其评分
3. 可视化：SC检索结果 + 最终配准

PROF

## 5. 性能预期

### 5.1 需求1（全局定位）

成功率：

- 地图覆盖良好区域: >90%
- 相似场景（长走廊）：70-80%（需多帧验证）

耗时：

|           |                   |
|-----------|-------------------|
| SC检索:     | ~50-100ms         |
| 局部地图提取:   | ~100-200ms × K个候选 |
| FPFH提取:   | ~200-500ms × K个候选 |
| TEASER配准: | ~100-300ms × K个候选 |
| 总计 (K=3): | 约2-4秒             |

## 5.2 需求2（局部优化）

成功率：

- 初值误差<10米: >95%
- 初值误差10-20米: >80%（需增大局部地图范围）

耗时：

|           |            |
|-----------|------------|
| 局部地图提取:   | ~100-200ms |
| FPFH提取:   | ~200-500ms |
| Quatro配准: | ~100-200ms |
| 总计:       | 约0.5-1秒    |

精度：

- 位置:  $\pm 0.05-0.1$ 米
- 姿态:  $\pm 0.5$ 度 (yaw)

---

## 6. 代码结构设计

### 6.1 Example 1 结构

PROF

```
example1_local_refinement/  
├─ main.cpp  
├─ local_refinement.h  
├─ local_refinement.cpp  
├─ utils.h (点云处理工具)  
└─ data/  
    ├─ map.ply  
    ├─ scan.pcd  
    └─ rough_pose.txt
```

核心函数：

```
class LocalRefinement {  
public:  
    // 主函数
```

```

bool refine(
    const PointCloud& map,
    const PointCloud& scan,
    const Pose& rough_pose,
    Pose& refined_pose
);

private:
    // 提取局部地图
    PointCloud extractLocalMap(
        const PointCloud& map,
        const Pose& center,
        double radius
    );

    // FPFH特征提取
    FPFHFeatures computeFPFH(
        const PointCloud& cloud,
        double normal_radius,
        double fpfh_radius
    );

    // Quatro配准
    bool performQuatro(
        const PointCloud& src,
        const PointCloud& tgt,
        const Correspondences& corr,
        Pose& result
    );
};

```

## 6.2 Example 2 结构

PROF

```

example2_global_localization/
├─ main.cpp
├─ global_localization.h
├─ global_localization.cpp
├─ scancontext_wrapper.h
├─ utils.h
└─ data/
    ├─ map.ply
    ├─ scan.pcd
    └─ sc_database.bin

```

核心函数：

```

class GlobalLocalization {
public:

```

```

// 主函数
bool localize(
    const PointCloud& map,
    const PointCloud& scan,
    Pose& estimated_pose
);

private:
// SC检索
std::vector<Candidate> retrieveCandidates(
    const PointCloud& scan,
    int top_k
);

// 验证单个候选
bool verifyCandidateWithQuatro(
    const Candidate& candidate,
    const PointCloud& map,
    const PointCloud& scan,
    Pose& result,
    double& quality_score
);

// 选择最优结果
Pose selectBestResult(
    const std::vector<VerificationResult>& results
);
};

```

## 7. 关键技术细节

### 7.1 局部地图提取策略

半径选择：

- 已知误差 + 扫描范围 + 安全余量
- 例如：
  - \* 误差3米 → 半径60米
  - \* 误差10米 → 半径80米

降采样：

- 地图：0.3-0.5米（减少计算量）
- 扫描：0.2米（保持特征）

### 7.2 特征匹配参数

FPFH：

- Airy稀疏点云 → 大邻域

```
* normal_radius: 1.0米  
* fpfh_radius: 2.0米
```

匹配:

- cross\_check: true (提高可靠性)
- ratio\_test: 0.8-0.9 (平衡精度和召回)

## 7.3 Quatro参数

```
teaser::RobustRegistrationSolver::Params params;  
params.noise_bound = 0.05; // Airy精度  
params.cbar2 = 1.0;  
params.estimate_scaling = false;  
params.rotation_estimation_algorithm = QUATRO;  
params.inlier_selection_mode = PMC_HEU;  
params.rotation_max_iterations = 100;  
params.rotation_gnc_factor = 1.4;
```

## 7.4 质量评估

配准质量指标:

1. Inlier数量 / 总对应关系数量
2. 配准残差 (RMSE)
3. 配准后点云重叠度

阈值:

- Inlier比例 > 30%
- RMSE < 0.15米
- 重叠度 > 50%

## 8. 测试方案

### 8.1 Example 1 测试

测试数据生成:

1. 从地图中随机选择一个位置
2. 提取该位置的"真实扫描" (模拟)
3. 添加噪声 ( $\pm 0.02$ 米)
4. 生成粗略位姿 (真实位姿 + 随机误差3-10米)
5. 运行算法
6. 对比估计位姿和真实位姿

### 测试场景：

- 开阔区域（特征丰富）
- 走廊（对称场景）
- 拐角（几何约束强）
- 不同误差等级（3米、5米、10米）

## 8.2 Example 2 测试

### 测试数据：

- 使用与Example 1相同的测试数据
- 但不提供粗略位姿
- 验证能否从零找到位置

### 评估指标：

- 定位成功率
- 定位精度
- 计算耗时
- 对相似场景的鲁棒性

---

## 9. 总结

### 需求1答案

#### TEASER/Quatro不能单独解决，需要配合Scan Context：

- SC负责全局搜索（在哪个区域）
- TEASER/Quatro负责精确定位（准确位置）

### 需求2答案

#### TEASER/Quatro完全可以解决：

- 3-10米误差对它来说不是问题
- 对outliers鲁棒
- 一次求解，不需要迭代

### 推荐方案

1. 先实现**Example 1**（需求2）- 验证TEASER/Quatro效果
2. 再实现**Example 2**（需求1）- 加入SC完成全局定位
3. **煤矿场景用Quatro**（4DOF）而非TEASER++（6DOF）

### 下一步

准备好后，我将为你编写：

1. Example 1: 局部精确定位（含完整代码）

## 2. Example 2: 全局定位（含完整代码）

你觉得这个方案如何？有什么需要调整的吗？